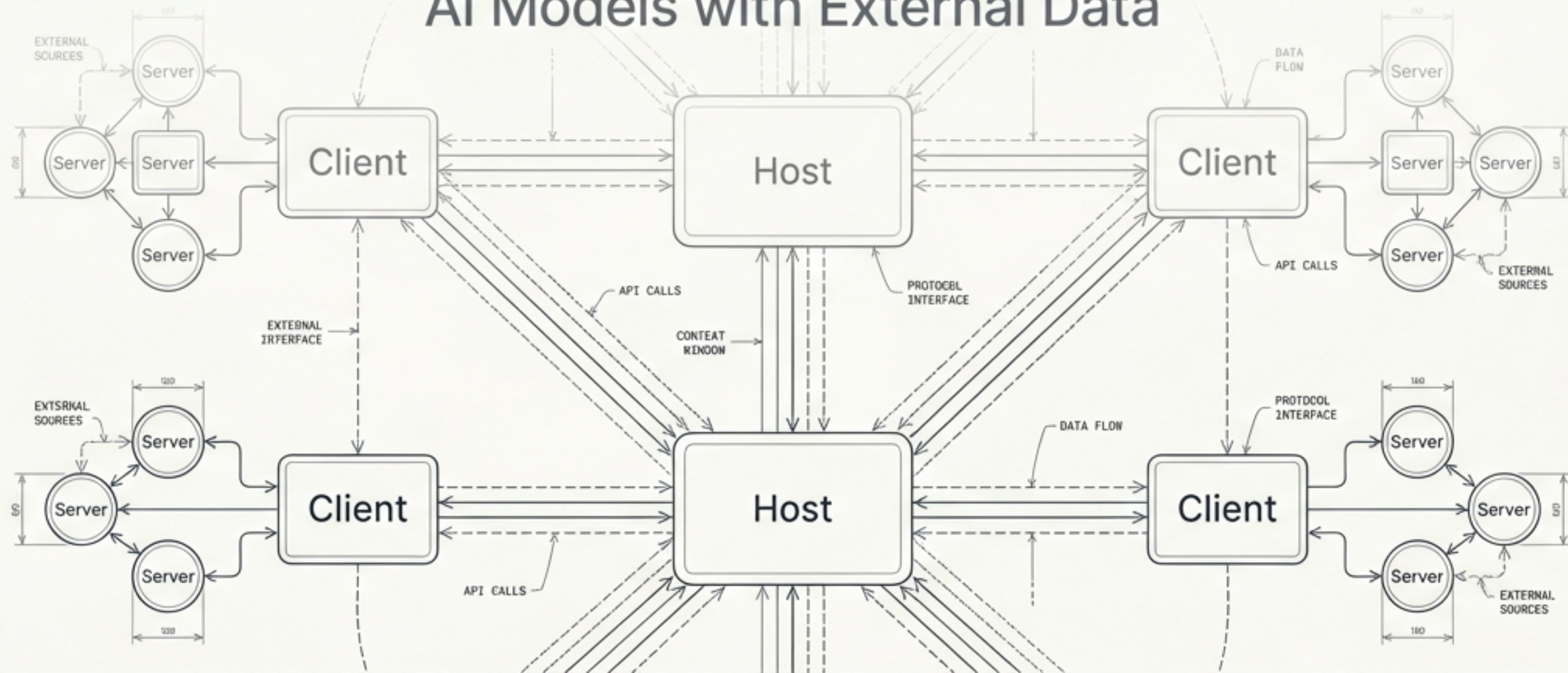


Anatomy of MCP: The Model Context Protocol

A First-Principles Deep Dive into Connecting
AI Models with External Data



The Core Foundation: Host and Server

The simplest version of the architecture involves only two entities attempting to communicate.

The Host (The AI): The application the user interacts with (e.g., Claude Desktop, Cursor). It houses the LLM.

The Server (The Tool):
A specialized tool executing tasks or accessing data (e.g., GitHub, Slack, Google Drive).

The Workflow:

User asks question -> Host creates prompt -> Host realizes it needs external data -> Host queries Server.

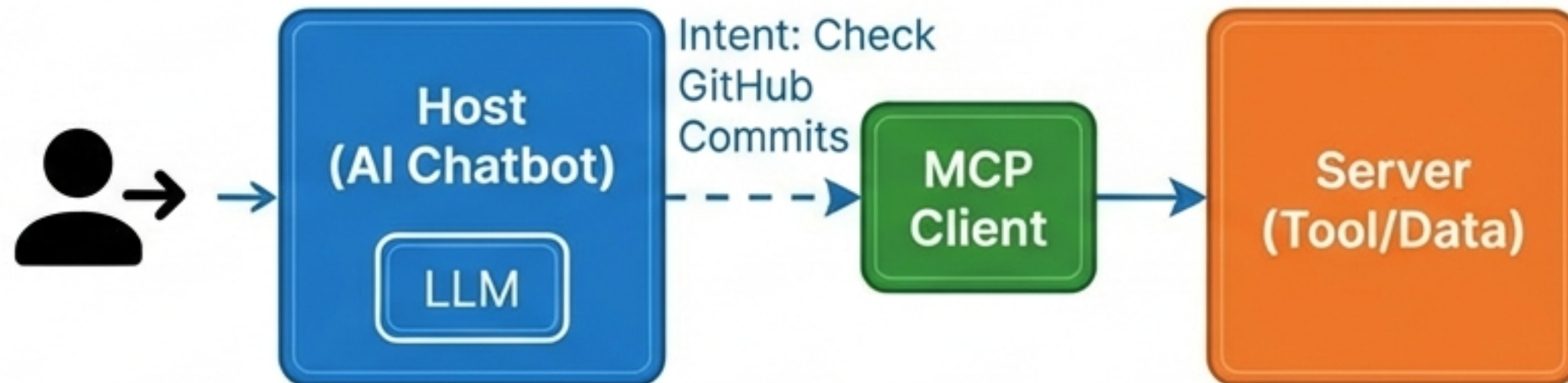


The Missing Link: The MCP Client

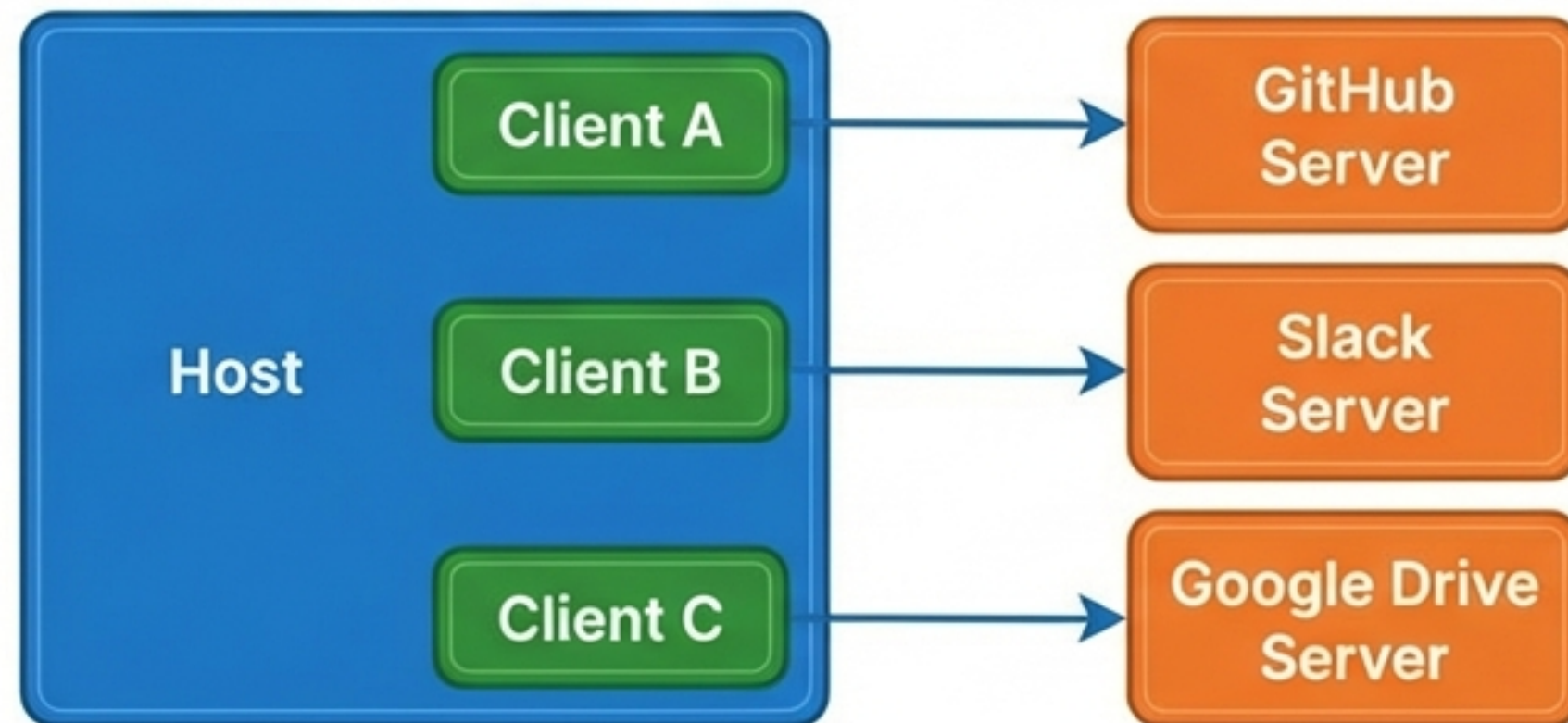
The Protocol Rule: Direct Host-to-Server communication is impossible. All interactions must be mediated.

The Client: A helper entity inside the Host that speaks the MCP language. It translates high-level AI requests into structured protocol messages.

The 1:1 Relationship: One Client connects to exactly one Server. To connect to both GitHub and Slack, the Host must instantiate two distinct Clients.

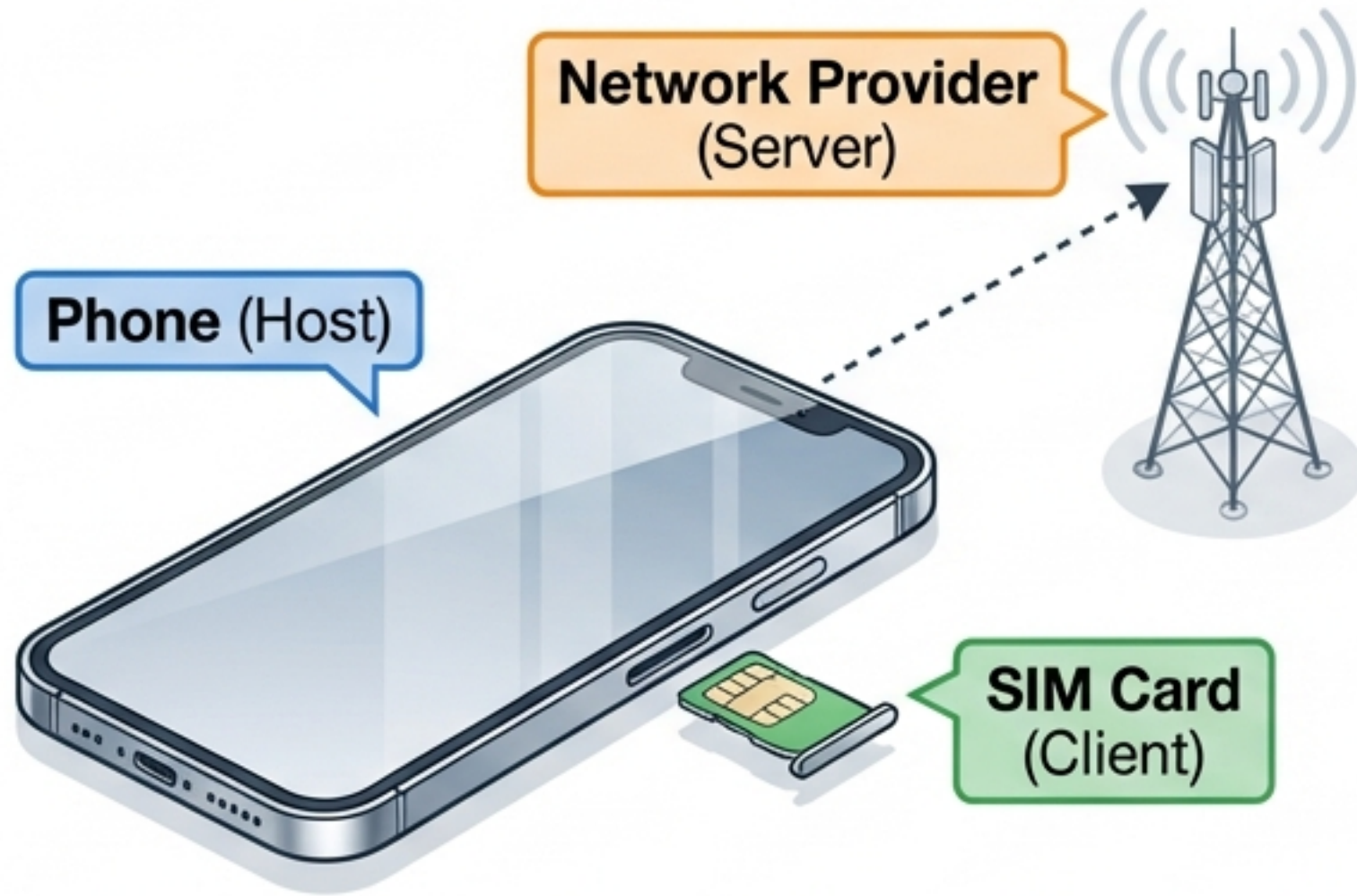


Multi-Server Setup



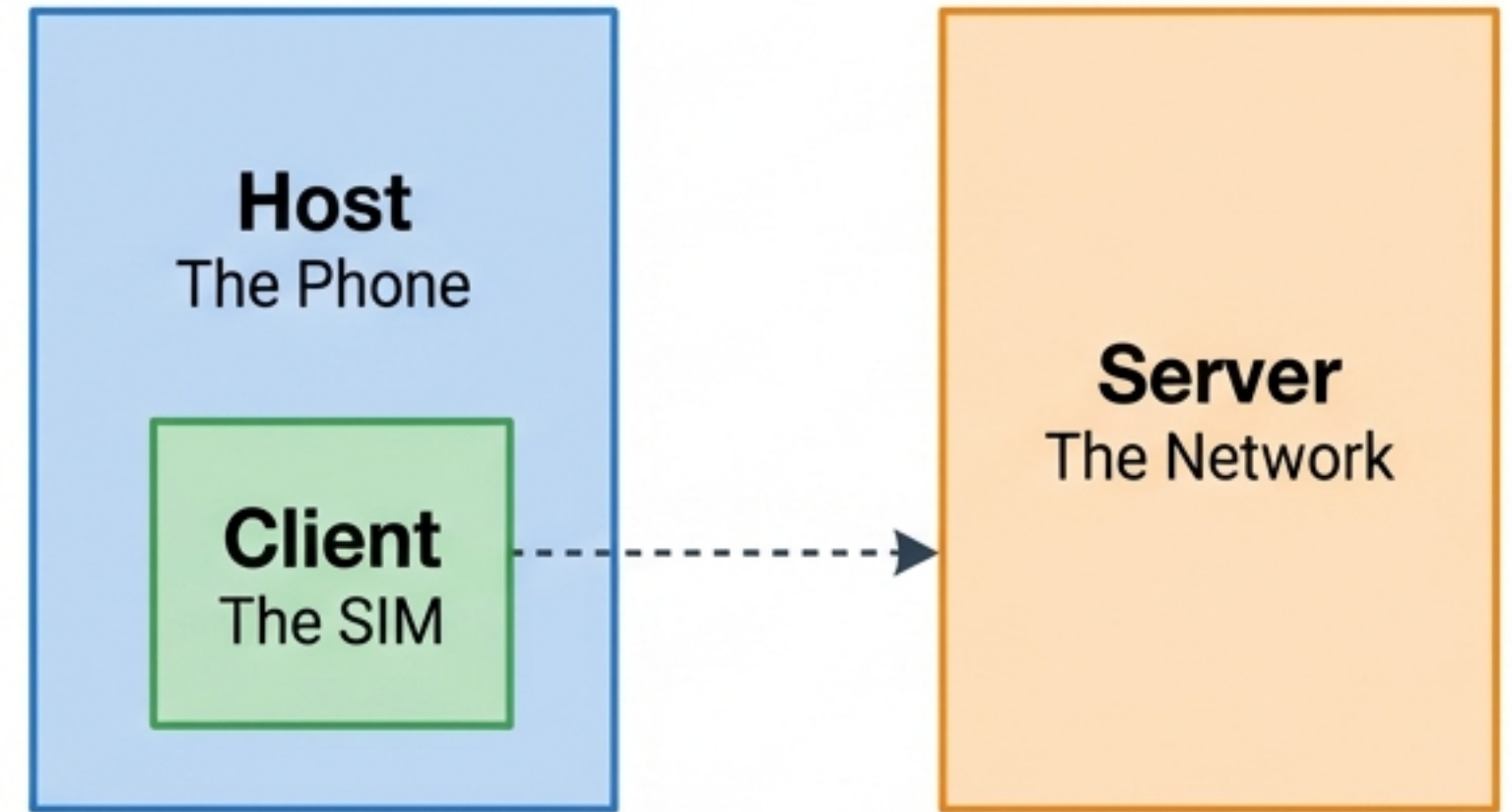
A Mental Model: The Phone Analogy

The Analogy



You need a specific SIM to access a specific Network.

The Architecture

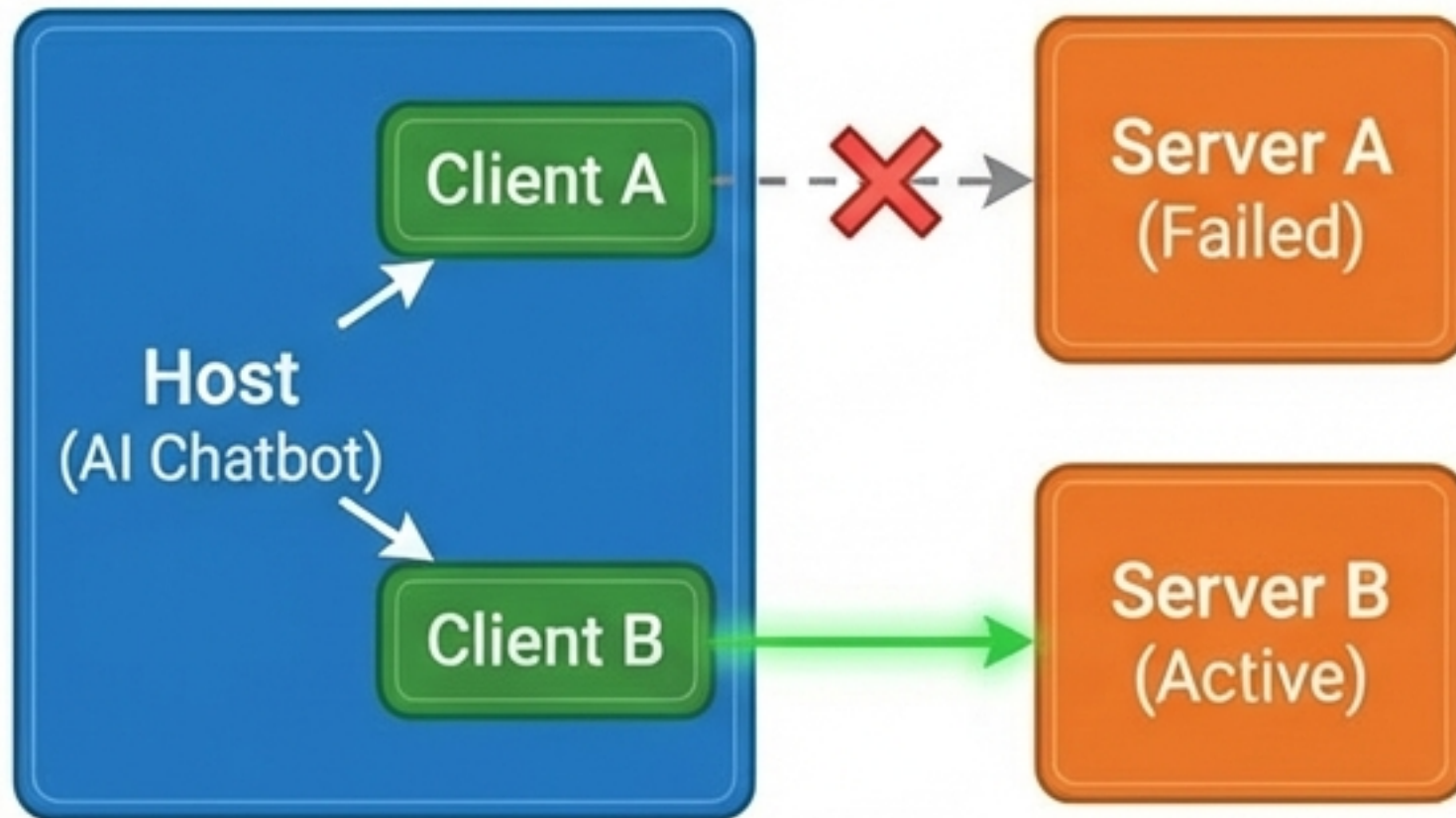


The Host needs a specific Client to access a specific Server.

Why This Architecture?

Decoupling & Safety

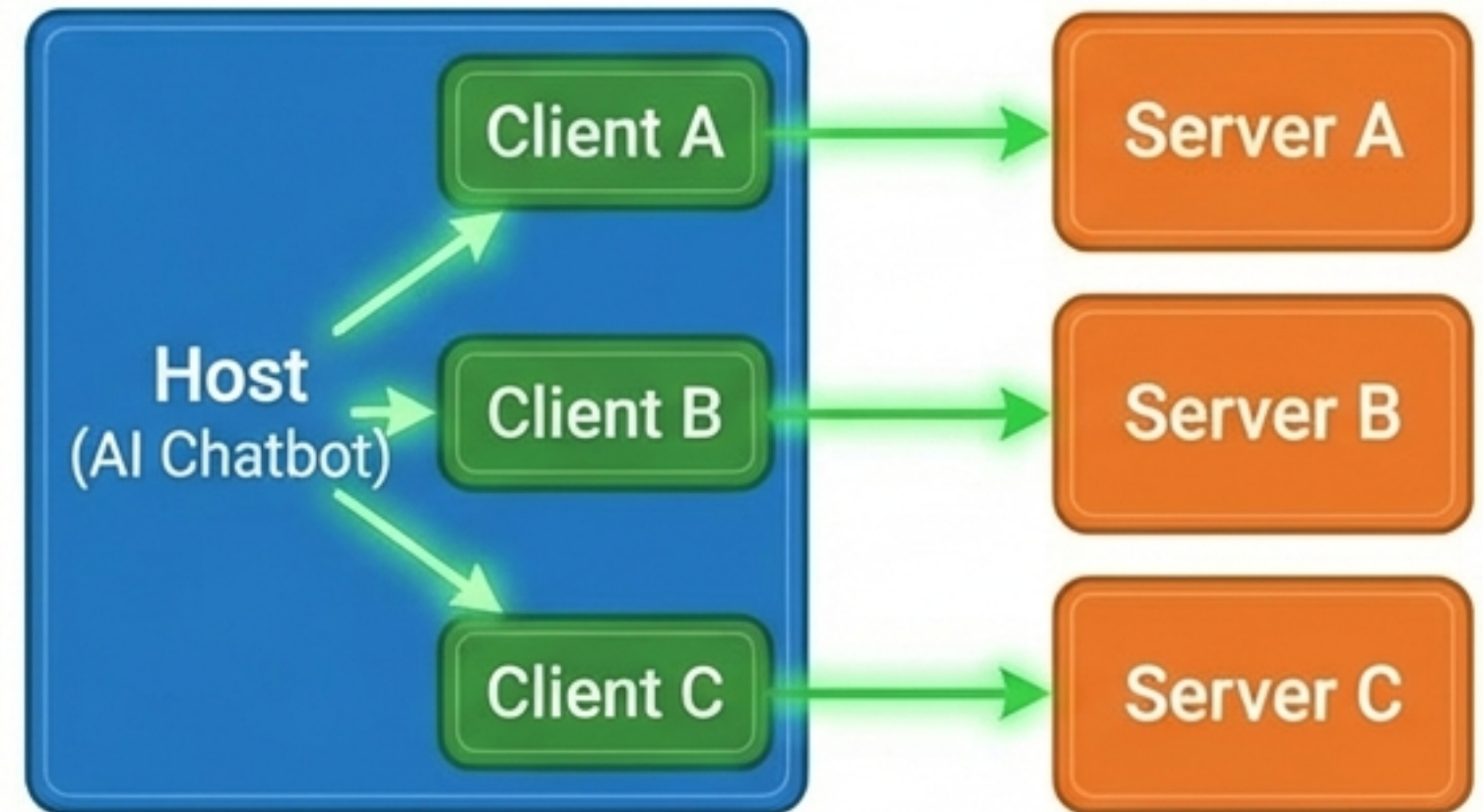
Separation of concerns acts as a firewall.
If one connection fails, others remain active.



Failure Isolation

Parallelism & Speed

Distinct Client instances allow the Host to
query multiple servers simultaneously.



Concurrent Execution

The Three Primitives

What does a Server actually offer to the Host?



TOOLS

Executable functions and dynamic actions.

Example: Create Issue, Post Message.



RESOURCES

Static data reading and structured content.

Example: Read File, Fetch Schema.



PROMPTS

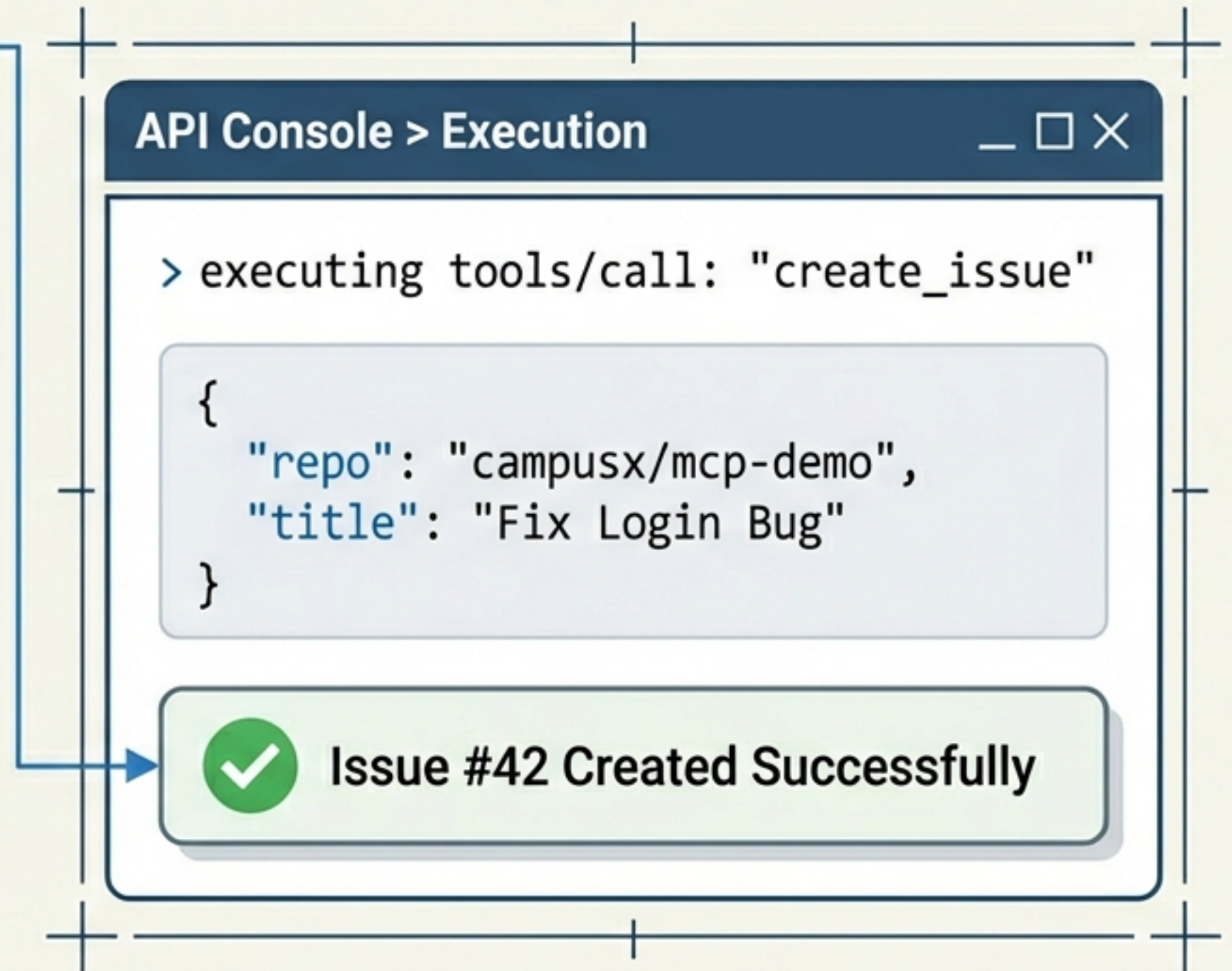
Pre-defined templates to guide AI behavior.

Example: Bug Report Template.

Swiss Engineering Manual

Primitive 1: Tools

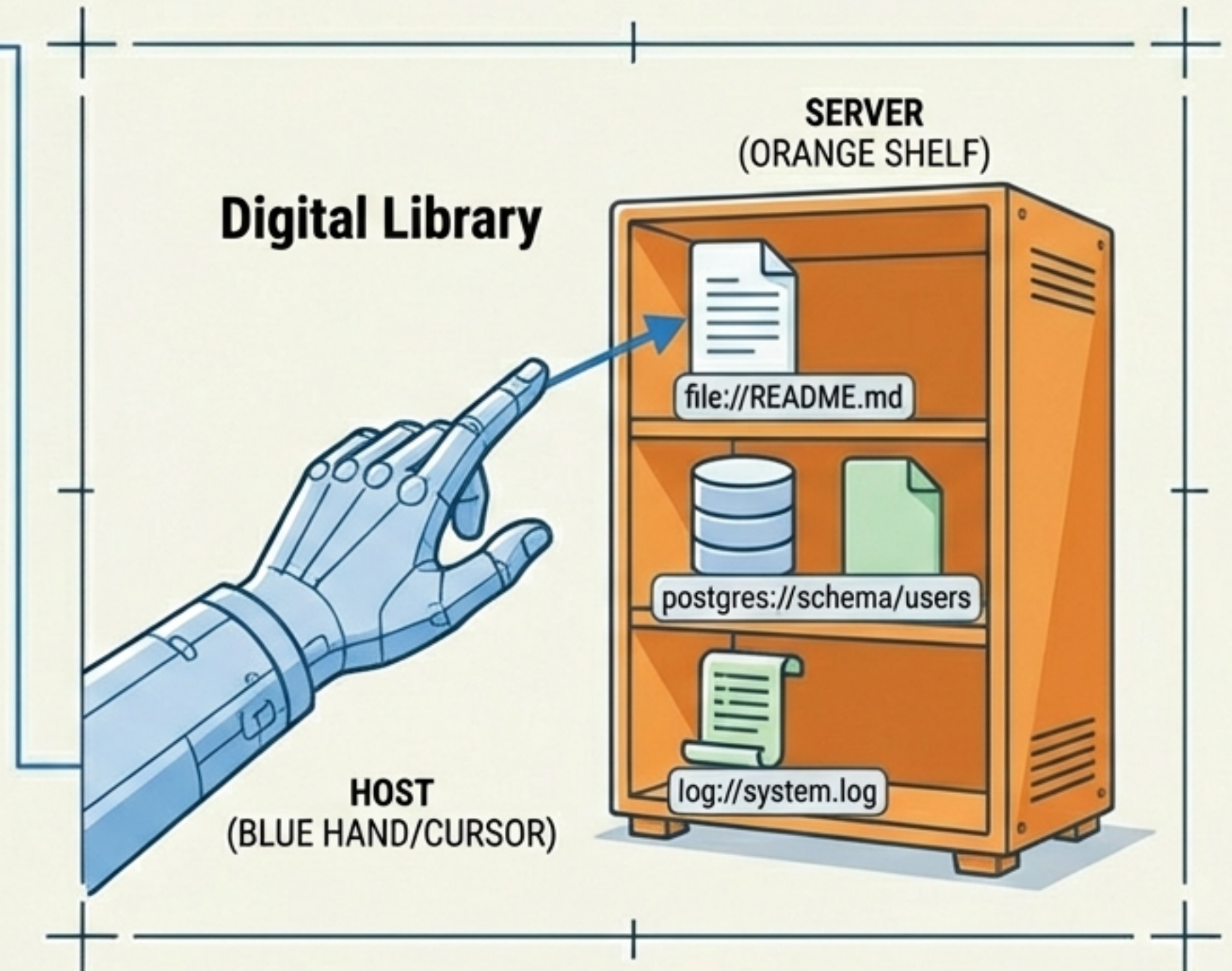
- **Definition:** Dynamic functions the AI can ask the Server to perform.
- **Key Operations:** `tools/list` (Discovery) and `tools/call` (Execution).
- **Examples:** `github_create_issue`, `drive_upload_file`, `slack_send_message`.



Primitive 2: Resources

Swiss Engineering Manual Digital Library Metaphor

- **Definition:** Access to structured, static data. Passive reading of information that doesn't change during access.
- **Key Operations:**
 - `resources/list`: What data do you have?
 - `resources/read`: Give me the content of this URI.
 - `resources/subscribe`: Notify me if this changes.



Primitive 3: Prompts

Solving the 'Vague Request' problem.

The Problem



Vague User Request.

Applied Prompt
Template

The Solution

A window with a dark blue header bar containing standard window controls (minimize, maximize, close). The main content area is white and contains the following structured text:

```
**Bug Report Template**  
**Title**: Login Button Unresponsive  
**Steps**: 1. Open App.  
                2. Click Login.  
**Expected**: Dashboard opens.  
**Actual**: Nothing happens.
```

Structured Server Input.

The Data Layer: JSON-RPC 2.0

- **Foundation:** A stateless, lightweight remote procedure call (RPC) protocol encoded in JSON.
- **Concept:** Executing a function on a different computer as if it were local.

Why JSON-RPC over REST?

- **Lightweight:** No heavy HTTP headers.
- **Transport Agnostic:** Works over HTTP, Stdio, WebSockets.
- **Bidirectional:** Servers can message Clients (Notifications).



Anatomy of a Message

****Client Request****

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "add",
    "args": {"a": 2, "b": 3}
  },
  "id": 1
}
```

****Server Response****

```
{
  "jsonrpc": "2.0",
  "result": {
    "content": 5
  },
  "id": 1
}
```

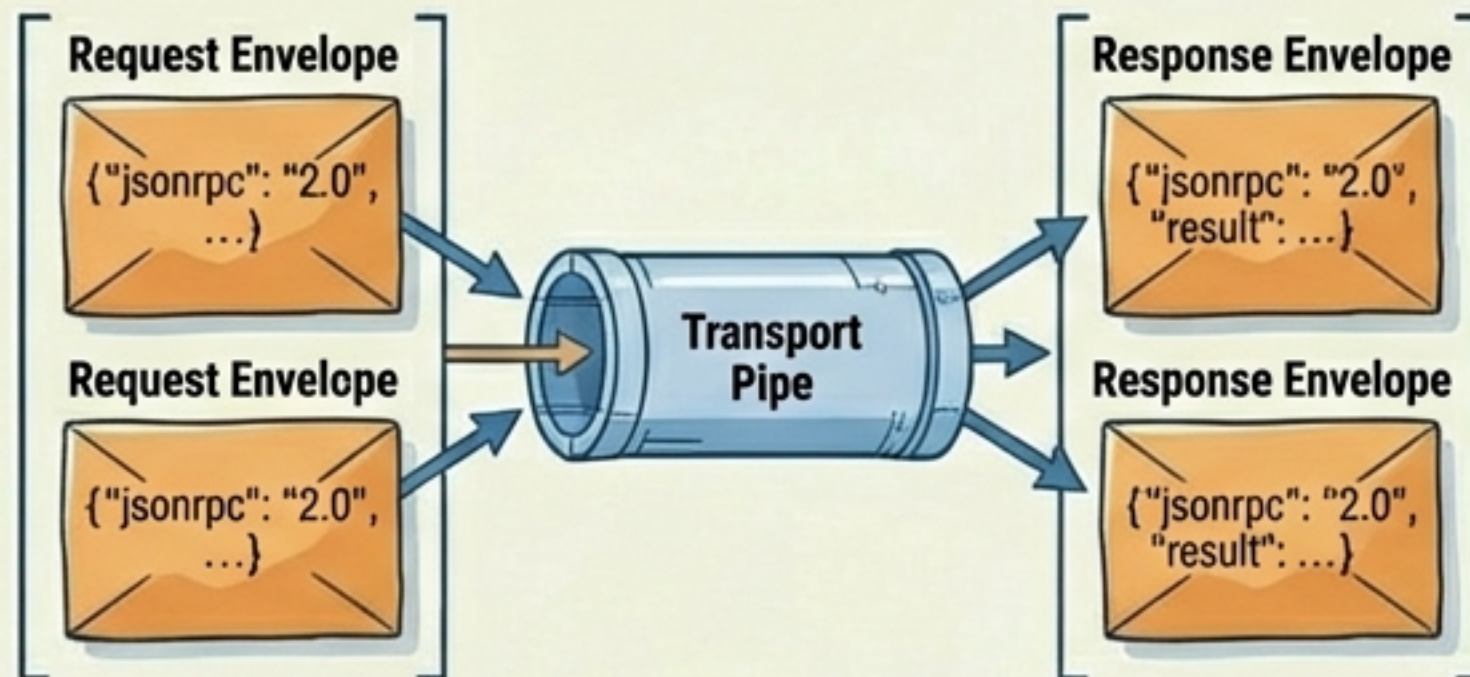
****Standardized Error****

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32601,
    "message": "Method not found"
  },
  "id": 1
}
```


Advanced Patterns: Batching & Notifications

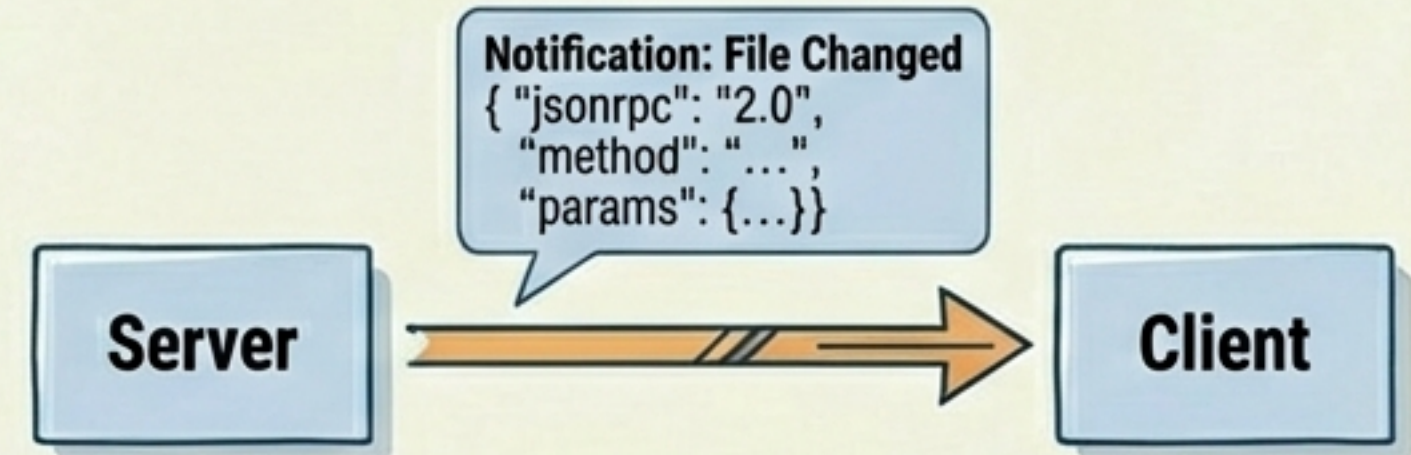
Batching

Sending multiple requests in a single array for efficiency.



Notifications

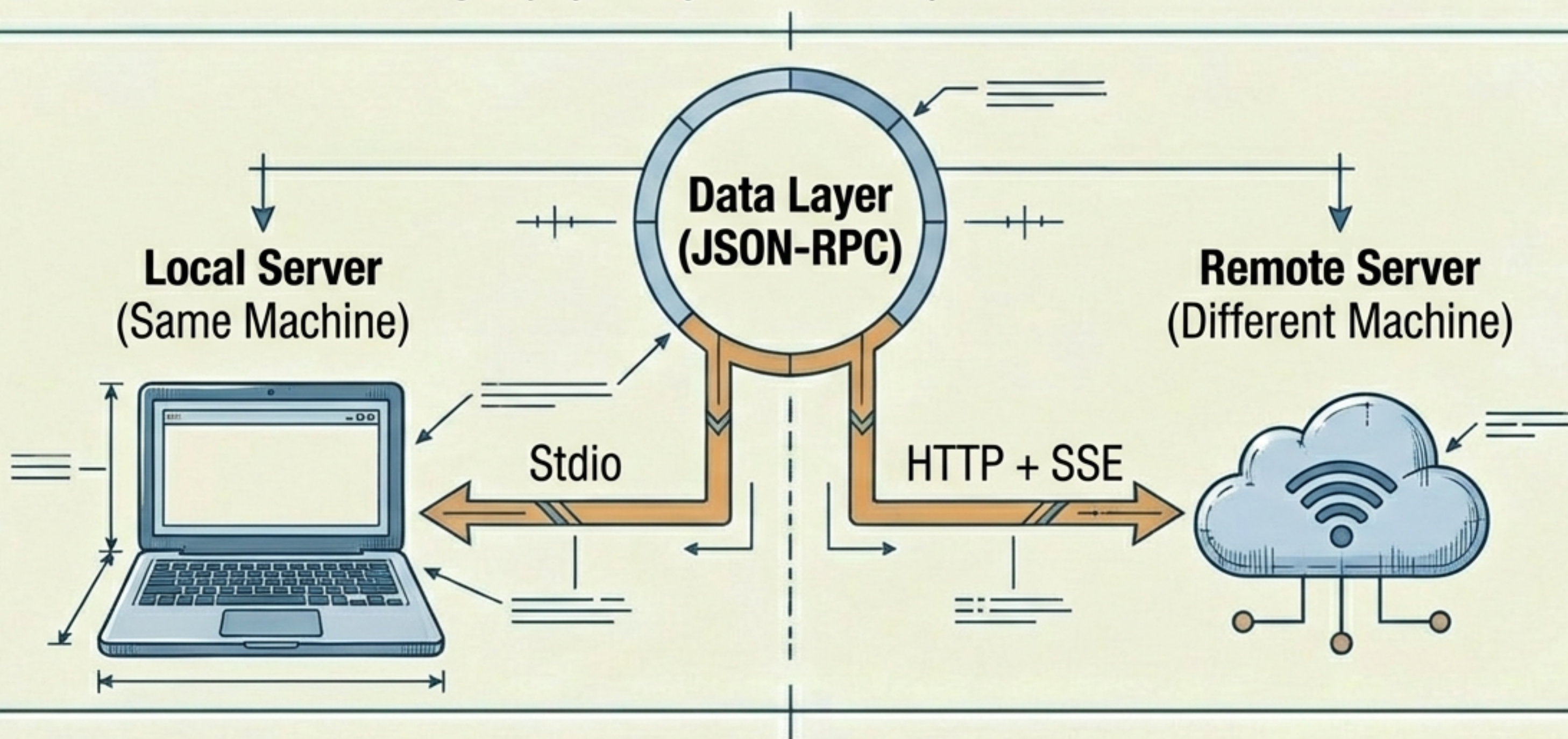
Fire and Forget. One-way messages with no 'id'.



Crucial for `resources/subscribe` updates.

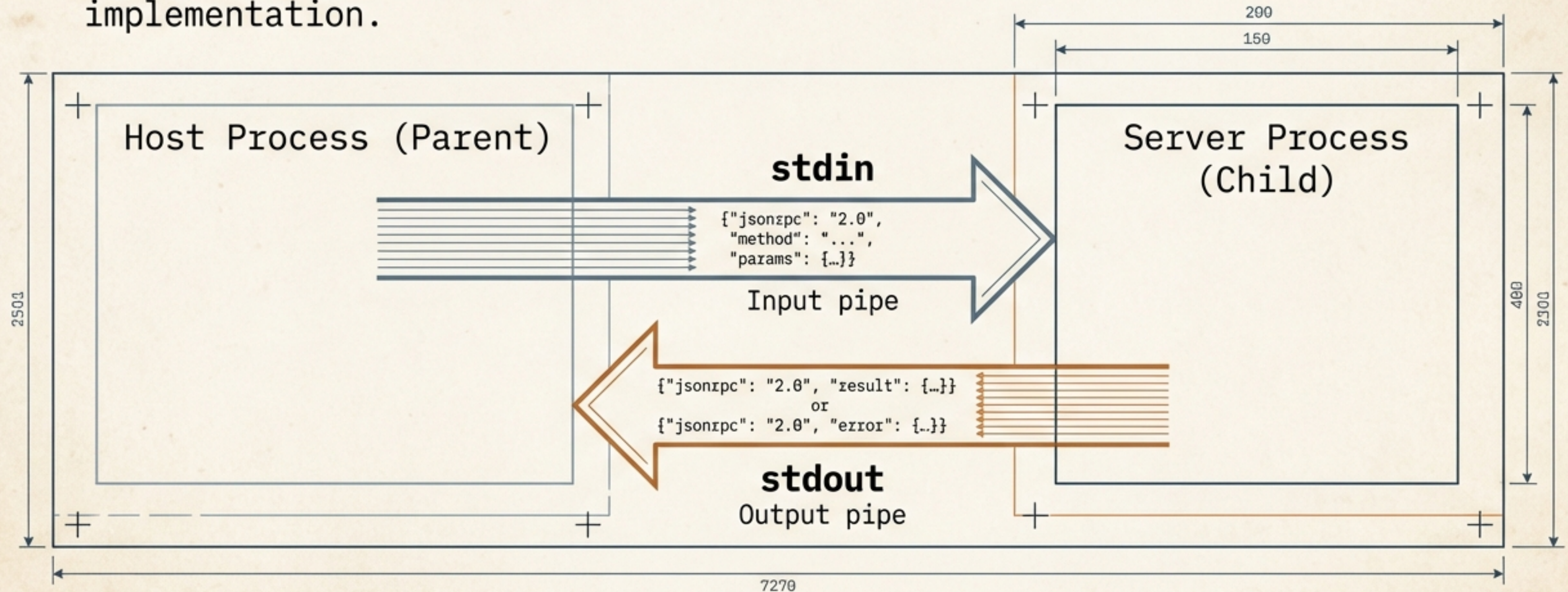
The Transport Layer: Two Paths

How do JSON messages physically move? It depends on where the server lives.



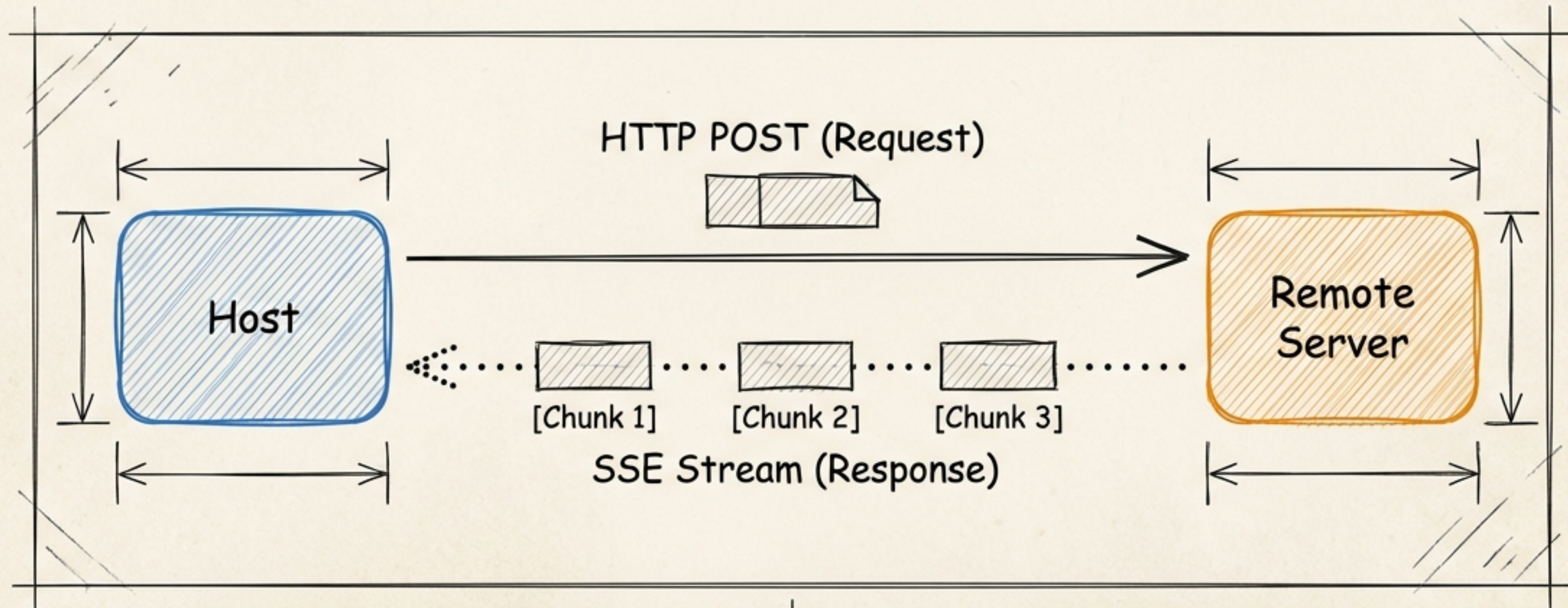
Local Transport: Stdio

- **Mechanism:** Standard Input/Output. Host launches Server as a Sub-process.
- **Benefits:** Low latency, high security (no ports), simple implementation.

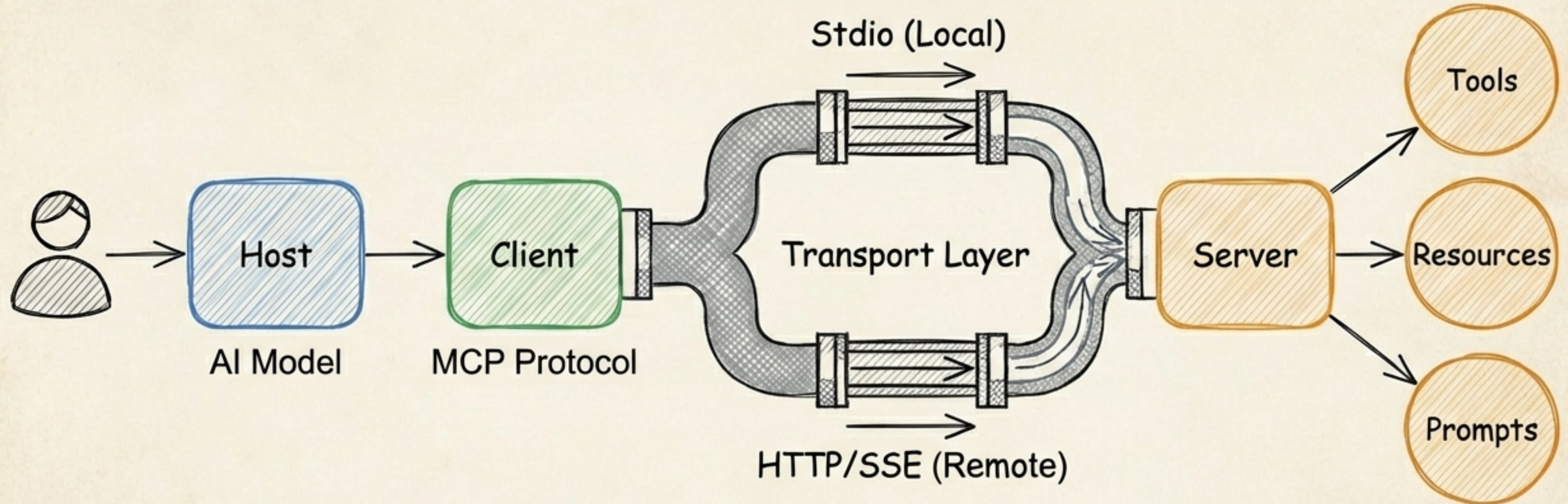


Remote Transport: HTTP + SSE

- **HTTP POST:** Used for sending commands (Client -> Server).
- **SSE (Server-Sent Events):** Used for receiving streaming responses (Server -> Client).
- **Why SSE?:** Enables "typing effect" and handles long-running AI tasks without timeouts.



The Complete MCP Architecture



Data Layer: JSON-RPC 2.0 flows through all connections.